

Voctomix 2.0

Desarrollo y optimización de un sistema modular de código abierto para la producción remota de vídeo en tiempo real

Martín Herranz Sánchez

Defensa de Trabajo Fin de Grado — 2026

Tutor: Álvaro Llorente Gómez

Escuela Técnica Superior de Ingenieros de Telecomunicación
Universidad Politécnica de Madrid

Contexto y motivación

Objetivos

Estado del arte

Arquitectura del sistema

Contribuciones

Despliegue

Resultados

Caso real: CyberNEMO

Conclusiones

Contexto y motivación

- La producción audiovisual en directo depende de mezcladores hardware de coste muy elevado.
- Necesidad de personal especializado y dependencia de fabricante.
- Las alternativas software (OBS, vMix) son monolíticas y de un solo operador.
- **Oportunidad**: democratizar la producción broadcast con software libre, contenedores y control programable.

Objetivos

Objetivos del trabajo

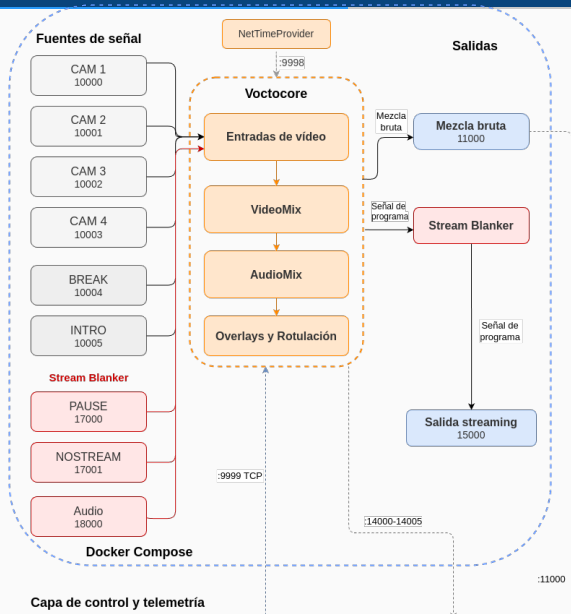
- Extender Voctomix con capacidades de producción profesional.
- Contenerizar el sistema completo para despliegue reproducible.
- Añadir telemetría en tiempo real para monitorización.
- Validar el sistema en múltiples escenarios de despliegue.

Estado del arte

- Mezcladores hardware (Blackmagic ATEM): fiables pero costosos.
- Software monolítico (OBS Studio, vMix): sin API ni modo headless.
- Modelo REMI: producción remota sobre IP.
- Voctomix v1.3: arquitectura cliente–servidor; desplegado como *Production Core* en un evento deportivo real.

Arquitectura del sistema

Arquitectura del sistema



- **votocore:** motor GStreamer (compositor, audio, salida).
- **votogui:** interfaz de control GTK 3.
- Separación estricta cliente-servidor sobre TCP.
- Telemetría desacoplada vía RabbitMQ.

Contribuciones

- Overlay PNG y dos capas de texto independientes.
- Fundido configurable (fade-in/fade-out).
- Desactivación automática en cada corte de fuente.

- Tres estados de salida: LIVE / PAUSE / NOSTREAM.
- Control de la señal de programa independiente de la mezcla interna.
- Transiciones limpias en frontera de buffer GStreamer.

- Fundido cruzado automático de audio al cambiar de fuente.
- Reduce la carga cognitiva del operador en directo.
- Independiente del modo de composición.

- Exporta el estado completo del mezclador como JSON.
- Eventos CHANGE (inmediatos) y STATE (periódicos) vía AMQP.
- Sin modificar el núcleo de la aplicación.

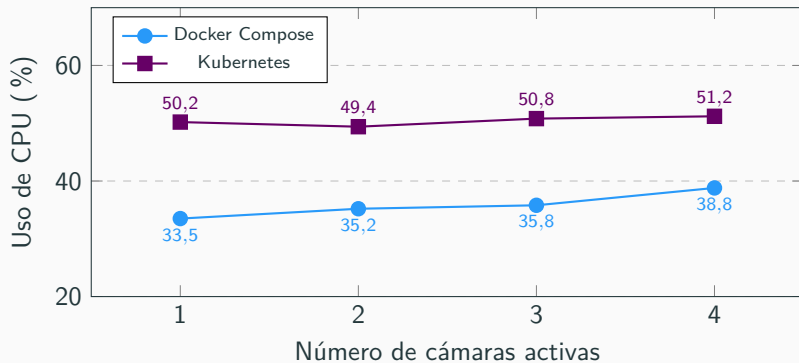
Despliegue

- Stack de 10 servicios desplegable con un único comando.
- `network_mode: service:votocore`: comunicación por loopback.
- Reproducibilidad total del entorno de ejecución.

- Pod multi-contenedor con voctocore y cámaras sidecar.
- RabbitMQ como StatefulSet con almacenamiento persistente.
- Orquestación y auto-recuperación declarativa.

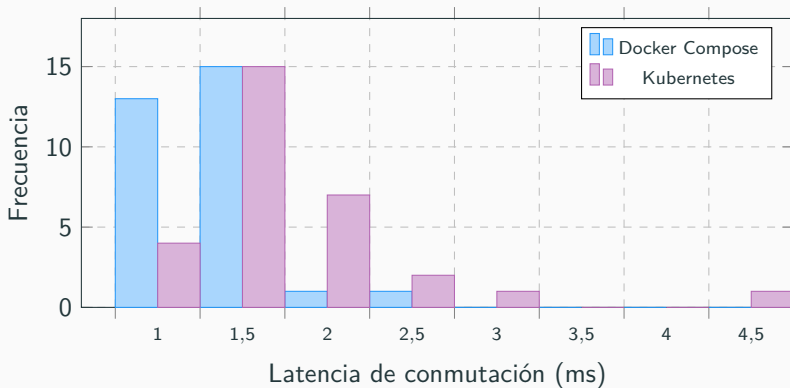
Resultados

Resultados — Consumo de CPU



En Docker la CPU escala de 33,5 % a 38,8 % con cuatro cámaras (+5,3 pp). El 50 % fijo en Kubernetes es el plano de control de Minikube en el mismo equipo.

Resultados — Latencia de conmutación



Mediana de **1,5 ms** (Docker) y 1,8 ms (K8s), muy por debajo del umbral de 45 ms del estándar ITU-R BT.1359-1. El cambio de cámara es imperceptible.

Caso real: CyberNEMO

- Uso real en el proyecto europeo CyberNEMO en la UPM.
- Sesiones de investigación y difusión con participantes remotos.
- Valida el sistema más allá de condiciones de laboratorio.

Conclusiones

- Voctomix 2.0 ofrece producción broadcast con software libre y contenedores, a coste de licencia cero.
- Validado en cuatro escenarios y en un despliegue real.
- **Líneas futuras:** cámaras físicas (DeckLink/NDI), salida SRT, interfaz web y monitorización Grafana.

Gracias por su atención

Áno de preguntas